Colibri روی جعبهٔ خانه — تست واقعی ۳۵B و ۷۴۴B بدون GPU

ادعای ریل: «مدل ۷۴۴ میلیارد پارامتری روی کامپیوتر معمولی» — این گزارش چیزی است که خودمان نصب و اندازه گرفتیم.

جعبه: ki · Ubuntu 26.04.1 LTS | Intel Core i5-10500T (6C/12T, 2.3-3.8 GHz) | ۳۰ GB RAM · SSD · بدون GPU

حکم: ادعا درست است

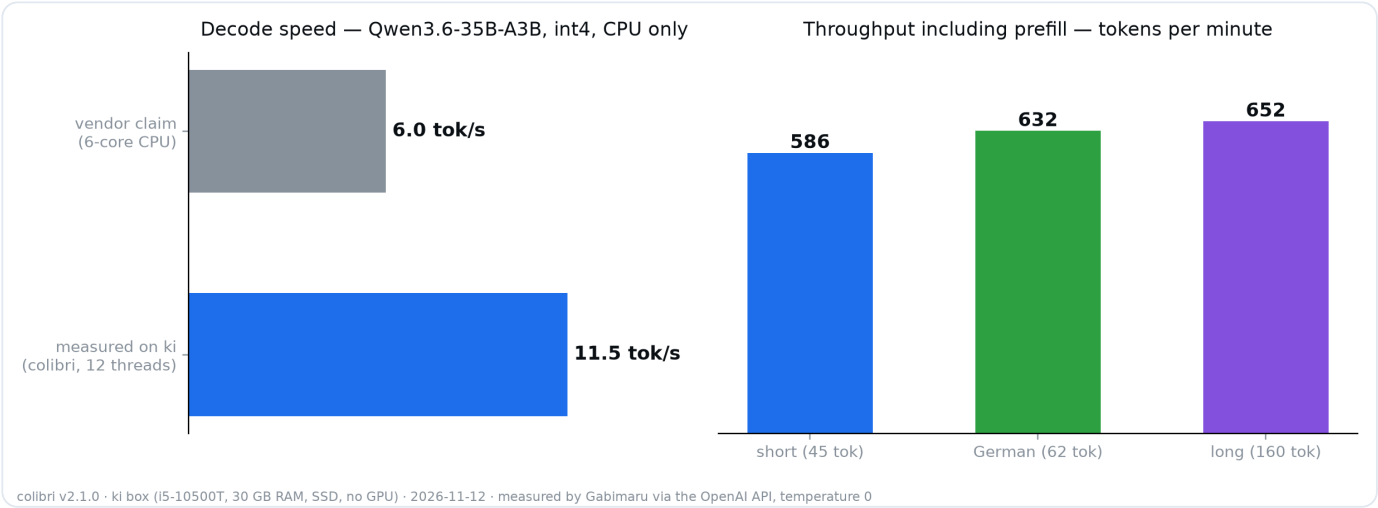
موتور **colibri v2.1.0** (Apache-2.0، نوشته‌شده با C) بخش dense مدل را در رم نگه می‌دارد و متخصص‌های routed را از SSD استریم می‌کند. روی این جعبه از مخزن رسمی نصب شد، با gcc 15.2 کامپایل شد و مدل Qwen3.6-35B-A3B در کانتینر int4 (۲۳ گیگ) از طریق API سازگار با OpenAI سرو شد. اعداد زیر از سه درخواست واقعی با `temperature 0` گرفته شده‌اند، نه از صفحهٔ پروژه.

سرعت تولید	توان عبوری	اولین توکن	اشغال حافظه
۱۱.۵	**۶۲۳**	**۰.۵۳**	**۱۴.۶**
توکن بر ثانیه (decode)	توکن در دقیقه (با prefill)	ثانیه (TTFT)	گیگابایت رم

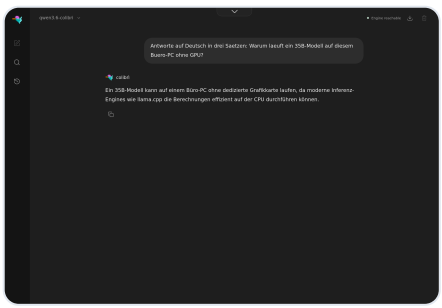
اندازه‌گیری واقعی روی ki — API، سه پرامپت

پرامپت	توکن ورودی	توکن خروجی	TTFT (ث)	tok/s	توکن/دقیقه	خواندن دیسک (MB)
`short_en`	۳۱	۴۵	۰.۷۲	**۱۱.۵۶**	۵۸۶	۵۴
`german`	۳۲	۶۲	۰.۵۳	**۱۱.۵۷**	۶۳۲	۵۴
`long_en`	۳۴	۱۶۰	۰.۷۷	**۱۱.۴۶**	۶۵۲	۱۵۶

مدل کامل در رم جا می‌شود، پس خواندن دیسک تقریباً صفر است؛ به همین دلیل سرعت از عدد اعلامی سازنده (۶ توکن/ثانیه روی CPU) بالاتر درآمد: **۱۱.۵** در برابر ۶.۰.



چپ: سرعت اندازه‌گیری‌شده در برابر ادعای سازنده. راست: توکن بر دقیقه برای سه پرامپت.



داشبورد زندهٔ همان سرور

سمت چپ: صفحهٔ چت colibri روی همین جعبه، متصل به مدل Qwen3.6-35B-A3B؛ پاسخ آلمانی در چند ثانیه تولید شد. همان موتور یک API سازگار با OpenAI روی پورت ۸۰۰۰ و یک صفحهٔ «نقشهٔ متخصص‌ها» دارد که نشان می‌دهد برای هر توکن کدام expertها از دیسک خوانده می‌شوند.

راهنمای نصب روی اوبونتو (همان کاری که انجام شد)

```
# 1) prerequisites - Ubuntu 24.04 / 26.04
sudo apt install -y git python3 build-essential
git clone --depth 1 https://github.com/JustVugg/colibri ~/colibri-moe/colibri
cd ~/colibri-moe/colibri

# 2) build the CPU engine (no GPU) - a few seconds
make -C c portable -j$(nproc)

# 3) which model fits this machine? (no download)
python3 c/coli setup --list

# 4) install a model + serve the OpenAI-compatible API
python3 c/coli setup --model qwen36-35b --yes
curl -s http://127.0.0.1:8000/v1/models
```

آنچه روی ki کار کرد: نصب و build با گ‌cc ۱۵٫۲ بدون خطا؛ مدل ۲۳ گیگی در حدود ۱۰ دقیقه دانلود شد (۳۷٫۷ مگابایت بر ثانیه) و API روی پورت ۸۰۰۰ بالا آمد.

تله: قبل از دانلود، `setup --list` را بزنید. معیار «رم»، نه دیسک است: بخش dense باید در رم جا شود، وگرنه موتور بالا نمی‌آید.

کدام مدل روی این جعبه جا می‌شود؟ (خروجی خود ابزار)

مدل	حجم دانلود	وضعیت	دلیل
`qwen36-35b`	GB 23	good	کل مدل در رم جا میشود (GB ۲۰)
`qwen3-coder-30b`	GB 19	good	کل مدل در رم (GB ۱۸)
`deepseek-v4-flash-reap`	GB 85	good	dense 7.9 GB در رم، expertها از SSD
`deepseek-v4-flash`	GB 167	good	dense 7.9 GB در رم
`mimo-v2.6-flash`	GB 172	tight	رم کم (GB ۳۲ از GB ۵۲)
`qwen38-flash-next`	GB 186	good	dense 8 GB در رم
`glm-5.3`	GB 419	good	dense 9.9 GB در رم

۱۱ خانوادهٔ مدل بررسی شد؛ `glm-5.2` با ۷۴۴ میلیارد پارامتر روی همین جعبه «good» است چون فقط ۹٫۹ گیگ از آن باید در رم بماند. `kimi-k3` (۱٫۵ تریلیون) با ۷۶۶ گیگ دیسک آزاد جا نمی‌شود.

و آن ۷۴۴ میلیارد پارامتر؟

سرعت تولید	توکن در دقیقه	اولین توکن	رم
۰.۰۳	۱.۴	۲۸۱.۹۶	۱۷.۵
توکن/ثانیه	با prefill	ثانیه	گیگابایت

مدل ۷۴۴ میلیارد پارامتری GLM-5.2 روی همان جعبه و بدون GPU: ۳۳ توکن در ۱,۳۷۸.۸ ثانیه، با ۶۱۷ گیگابایت خواندن از دیسک در همان درخواست (حدود ۱۸.۷ گیگ برای هر توکن) — یعنی متخصص‌ها واقعاً از SSD می‌آیند. یعنی «اجرا می‌شود» ولی «قابل استفاده» نیست: هر توکن حدود ۳۳ ثانیه.

چه چیزی و کجا ذخیره شد

مورد	مسیر / لینک
اسکیل	`skills/devops/colibri-local-moe`
اسکریپت بنچ	`workspace/colibri-moe/bench_api.py`
دادهٔ خام + این PDF	`github.com/ahmadvahidi/toolbox · docs/reports/`